#9 구역 미리보기 계산 — 코드 상세 설명서

출처: Notion `[#9] 코드리뷰 · 구역 미리보기 계산` (PR #105). 아래 코드는 Notion에 적힌 초안 코드이며, 현재와 다른 점은 각 항목에 표시했다.

1. 전체 흐름

```
브라우저(그린 도형 GeoJSON)
   → ZoneApiController      접수원: 요청 받기
   → ZonePreviewService     창구장: 순서대로 시키기
        ├→ ZonePolygonReader      검사원: 도형이 올바른가
        ├→ RegionRepository + RegionBoundaryReader   지번 담당: 어느 동 안에 있나
        └→ ZoneAreaCalculator     측량사: 위도 보정해서 ㎡ 계산
   → 응답 {areaM2, dongId, dongName}
```

역할 (비유)	파일	하는 일
접수원	`ZoneApiController`	`POST /api/v1/manage/zones/preview` 요청을 받아 서비스에 넘김
창구장	`ZonePreviewService`	검사 → 동 찾기 → 넓이 계산을 순서대로 실행
검사원	`ZonePolygonReader`	GeoJSON을 도형으로 바꾸고 올바른지 검사
측량사	`ZoneAreaCalculator`	위도·경도를 미터로 바꿔 넓이 계산
지번 담당	`RegionBoundaryReader`, `RegionRepository.findByLevel`	DB에 저장된 동 경계를 도형으로 읽어 오기
서류 양식	`ZonePreviewRequest`, `ZonePreviewResponse`	요청·응답 모양
경보	`InvalidPolygonException`, `OutOfServiceAreaException`	"도형이 이상해요", "서비스 지역 밖이에요"
증거	`ZonePreviewTest`	자동 테스트 9개

관리자 앱의 `context-path`가 `/admin`이라 실제 주소 앞에 `/admin`이 붙는다. GeoJSON 좌표는 **[경도, 위도]** 순서다. (#7과 같다)

2. 요청과 응답

```
POST /admin/api/v1/manage/zones/preview
{"boundary": {"type": "Polygon", "coordinates": [[[127.001, 37.601], [127.003, 37.601], [127.003, 37.
603], [127.001, 37.603], [127.001, 37.601]]]}}

200 {"areaM2": 2481.3, "dongId": 3, "dongName": "궁동"}      ← 명세 예시값
400 {"code": "INVALID_POLYGON", "message": "..."}
400 {"code": "OUT_OF_SERVICE_AREA", "message": "..."}
```

3. 코드

3-1. 동 목록 조회: RegionRepository (한 줄 추가)

```
public interface RegionRepository extends JpaRepository<Region, Long> {

    List<Region> findByCodeIn(Collection<String> codes);

    List<Region> findByLevel(RegionLevel level);
}
```

"단위가 이것인 지역을 전부 달라"는 주문서다. 스프링 데이터 JPA가 메서드 이름(`findBy` + `Level`)을 읽어 SQL을 만든다.

개정 반영: #7 리뷰로 `RegionLevel.DONG`이 `EMD`로 바뀌어서, 이 메서드를 부르는 곳은 `findByLevel(RegionLevel.EMD)`가 된다.

3-2. 서류 양식: ZonePreviewRequest, ZonePreviewResponse

```
/** boundary는 GeoJSON Polygon. 모양 검증은 service가 한다(INVALID_POLYGON). */
public record ZonePreviewRequest(@NotNull(message = "필수입니다.") JsonNode boundary) {}

public record ZonePreviewResponse(double areaM2, Long dongId, String dongName) {}
```

- `record` : 한 번 만들면 값을 못 바꾸는 상자. 요청·응답은 DTO로 분리하라는 팀 규칙이다.
- `JsonNode` : JSON을 "칸 안의 칸"으로 받는 그릇. boundary를 그대로 받아 검사원이 자세히 본다.
- `@NotNull` : boundary가 아예 없으면 컨트롤러 단계에서 막는다.

3-3. 경보: InvalidPolygonException, OutOfServiceAreaException

```
public class InvalidPolygonException extends RuntimeException {
    public InvalidPolygonException(String message) { super(message); }
    public InvalidPolygonException(String message, Throwable cause) { super(message, cause); }
}

public class OutOfServiceAreaException extends RuntimeException {
    public OutOfServiceAreaException(String message) { super(message); }
}
```

개정 반영: 현재는 두 예외가 `BusinessException`을 상속하고, `ErrorCode`에 `INVALID_POLYGON`, `OUT_OF_SERVICE_AREA`가 등록돼 있다. 그래서 공통 오류 처리가 알아서 400 응답으로 바꿔 준다. 이름만 보고 원인이 보이고, 명세의 오류 코드와 1:1로 대응한다.

3-4. 검사원: ZonePolygonReader (핵심 ①)

```
/** 요청의 GeoJSON Polygon을 JTS 다각형으로 바꾸고 유효한지 확인한다(FNC-A-013 처리 순서 1). */
@Component
public class ZonePolygonReader {

    private final GeometryFactory geometryFactory = new GeometryFactory();

    public Polygon read(JsonNode boundary) {
        if (boundary == null || !"Polygon".equals(boundary.path("type").asString(""))) {
            throw new InvalidPolygonException("다각형(Polygon)이 아닙니다.");
```

```
        }
        JsonNode rings = boundary.path("coordinates");
        if (!rings.isArray() || rings.isEmpty()) {
            throw new InvalidPolygonException("다각형 좌표가 없습니다.");
        }
        try {
            LinearRing shell = toRing(rings.get(0));
            LinearRing[] holes = new LinearRing[rings.size() - 1];
            for (int i = 1; i < rings.size(); i++) {
                holes[i - 1] = toRing(rings.get(i));
            }
            Polygon polygon = geometryFactory.createPolygon(shell, holes);
            // 점 3개 이상(닫힌 고리 4점)은 LinearRing 생성에서, 자기 교차는 isValid에서 걸러진다
            if (polygon.isEmpty() || !polygon.isValid()) {
                throw new InvalidPolygonException("다각형이 올바르지 않습니다.");
            }
            return polygon;
        } catch (IllegalArgumentException e) {
            throw new InvalidPolygonException("다각형 좌표가 올바르지 않습니다.", e);
        }
    }

    private LinearRing toRing(JsonNode positions) {
        if (!positions.isArray()) {
            throw new InvalidPolygonException("다각형 좌표 형식이 올바르지 않습니다.");
        }
        Coordinate[] coordinates = new Coordinate[positions.size()];
        for (int i = 0; i < coordinates.length; i++) {
            JsonNode position = positions.get(i);
            if (!position.isArray()
                    || position.size() < 2
                    || !position.get(0).isNumber()
                    || !position.get(1).isNumber()) {
                throw new InvalidPolygonException("좌표 형식이 올바르지 않습니다.");
            }
            // GeoJSON 순서는 [경도, 위도]
            coordinates[i] = new Coordinate(position.get(0).asDouble(), position.get(1).asDouble());
        }
        return geometryFactory.createLinearRing(coordinates);
    }
}
```

검사 순서

단계	무엇을 검사하나
① 종류	`type`이 `Polygon`인가 (Point, MultiPolygon 등은 거부)
② 좌표 목록	`coordinates`가 비어 있지 않은 목록인가
③ 점 하나하나	`[경도, 위도]` 숫자 두 개인가
④ 닫힌 고리	점이 4개 이상이고 처음과 끝이 같은가. 아니면 JTS가 `IllegalArgumentException`을 던지고 우리 에러로 바꾼다
⑤ 꼬임 검사	`isValid()`: 선이 자기 자신을 가로지르는 나비넥타이 모양이면 거부

줄별 해설

코드	쉬운 뜻
`rings.get(0)` / `i = 1`부터	GeoJSON은 첫 고리가 바깥 테두리, 그 뒤 고리는 구멍

코드	쉬운 뜻
`createLinearRing(...)`	점들을 이어 닫힌 선을 만든다. 닫혀 있지 않거나 점이 모자라면 에러
`polygon.isValid()`	JTS의 도형 유효성 검사(자기 교차, 구멍이 바깥으로 나감 등)
`catch (IllegalArgumentException e)`	JTS의 "도형이 될 수 없는 좌표" 에러를 잡아 `INVALID_POLYGON`으로 바꿈
`new Coordinate(경도, 위도)`	JTS 점. x=경도, y=위도 순서

3-5. 측량사: ZoneAreaCalculator (핵심 ②)

```
@Component
public class ZoneAreaCalculator {

    private static final double METERS_PER_DEGREE_LNG_AT_EQUATOR = 111_320.0;
    private static final double METERS_PER_DEGREE_LAT = 110_540.0;

    public double areaM2(Polygon polygon, Point interiorPoint) {
        double lngScale = METERS_PER_DEGREE_LNG_AT_EQUATOR * Math.cos(Math.toRadians(interiorPoint.ge
tY()));
        Geometry meters = polygon.copy();
        meters.apply(new CoordinateSequenceFilter() {
            @Override
            public void filter(CoordinateSequence sequence, int i) {
                sequence.setOrdinate(i, Coordinate.X, sequence.getX(i) * lngScale);
                sequence.setOrdinate(i, Coordinate.Y, sequence.getY(i) * METERS_PER_DEGREE_LAT);
            }

            @Override
            public boolean isDone() { return false; }

            @Override
            public boolean isGeometryChanged() { return true; }
        });
        return meters.getArea();
    }
}
```

도형의 **모든 점의 좌표를 미터로 바꾼 복사본**을 만들고, 그 복사본의 평면 넓이를 `getArea()`로 구한다.

- `polygon.copy()` : 원본은 그대로 두고 복사본만 바꾼다. 원본 좌표는 동 찾기에서 계속 쓰기 때문이다.
- `Math.cos(Math.toRadians(위도))` : cos 함수는 라디안을 쓰므로 도를 먼저 바꾼다.
- 구멍이 있는 도형이면 구멍 넓이는 JTS가 알아서 뺀다.
- `111_320.0` : 숫자 사이의 `_`는 읽기 편하라고 넣은 구분자다.
- `CoordinateSequenceFilter` : 도형의 점을 하나씩 돌며 값을 바꾸는 JTS 도구. `isDone()`이 false라 모든 점을 돈다. `isGeometryChanged()`가 true라 JTS가 계산 캐시를 새로 만든다.

3-6. 창구장: ZonePreviewService (핵심 ③)

```
/** 그린 다각형의 면적·소속 동을 저장 전에 계산한다(FNC-A-013). 구역 등록(FNC-A-014)이 같은 계산을 다시 쓴다. */
@Service
@RequiredArgsConstructor
public class ZonePreviewService {

    private static final int AREA_SCALE = 1;

    private final RegionRepository regionRepository;
    private final ZonePolygonReader polygonReader;
```

```
    private final ZoneAreaCalculator areaCalculator;
    private final RegionBoundaryReader boundaryReader;

    @Transactional(readOnly = true)
    public ZonePreviewResponse preview(JsonNode boundary) {
        Polygon polygon = polygonReader.read(boundary);
        Point interiorPoint = polygon.getInteriorPoint();
        Region dong = findDongContaining(interiorPoint);
        double areaM2 = BigDecimal.valueOf(areaCalculator.areaM2(polygon, interiorPoint))
                .setScale(AREA_SCALE, RoundingMode.HALF_UP)
                .doubleValue();
        return new ZonePreviewResponse(areaM2, dong.getId(), dong.getName());
    }

    private Region findDongContaining(Point point) {
        for (Region dong : regionRepository.findByLevel(RegionLevel.DONG)) {   // 현재는 EMD
            if (boundaryReader.read(dong).contains(point)) {
                return dong;
            }
        }
        throw new OutOfServiceAreaException("서비스 지역 밖입니다.");
    }
}
```

직접 계산하지 않고 **순서대로 시키는 사람**이다.

1. 검사원이 도형을 검사한다. (실패하면 `INVALID_POLYGON`)
2. 도형 안쪽 점을 구한다(`getInteriorPoint`). 이 점이 "이 구역의 대표 위치"다.
3. 모든 동 경계를 하나씩 보며 그 점이 **안에 들어 있는 동**을 찾는다. 없으면 `OUT_OF_SERVICE_AREA`.
4. 같은 점의 위도로 넓이를 계산하고 소수 첫째 자리로 반올림한다. (명세 예시 `2481.3`처럼)

코드	쉬운 뜻
`@Transactional(readOnly = true)`	읽기만 한다는 표시. 저장은 안 한다
`@RequiredArgsConstructor`	`final` 필드를 받는 생성자를 롬복이 자동 작성(생성자 주입 규칙을 짧게 지키는 방법)
`BigDecimal.valueOf(x).setScale(1, HALF_UP).doubleValue()`	소수 첫째 자리까지 반올림한 뒤 다시 숫자로. 부동소수점 오차 없이 반올림하려고 BigDecimal을 거친다
`boundary.contains(point)`	"이 도형이 그 점을 안에 품고 있나"
`for ... if ... return ... throw`	하나씩 확인하다 찾으면 바로 돌려주고, 끝까지 없으면 에러

`preview()` 하나로 모아 둔 이유는 #10 구역 등록이 명세상 "이 계산을 다시 수행"해야 해서 그대로 재사용하려는 것이다.

3-7. 지번 담당: RegionBoundaryReader + readGeometry

```
/** region.boundary(GeoJSON 문자열)를 JTS 도형으로 읽는다. 적재 때 검증을 거친 값이라 읽기 실패는 데이터 이상이다. */
@Component
@RequiredArgsConstructor
public class RegionBoundaryReader {

    private final RegionGeoJsonParser geoJsonParser;

    public Geometry read(Region region) {
        try {
            return geoJsonParser.readGeometry(region.getBoundary(), region.getCode());
        } catch (RegionFileFormatException e) {
            throw new IllegalStateException("저장된 경계를 읽을 수 없습니다: " + region.getCode(), e);
```

```
        }
    }
}
```

```
// RegionGeoJsonParser.java (#7 파일에 메서드 하나 추가)
public Geometry readGeometry(String geometryJson, String code) {
    try {
        return toGeometry(jsonMapper.readTree(geometryJson), code);
    } catch (JacksonException e) {
        throw new RegionFileFormatException("경계를 읽을 수 없습니다: " + code, e);
    }
}
```

DB에 글자(JSON)로 저장된 동 경계를 **다시 도형으로 되살리는** 역할이다.

- #7의 검수원이 이미 Polygon/MultiPolygon을 읽는 코드를 갖고 있어서 새로 만들지 않고 **그대로 재사용**했다. (공개 메서드 하나만 추가, 기존 동작은 그대로)
- 처음엔 JTS의 `GeoJsonReader`를 쓰려 했지만 별도 라이브러리(`jts-io-common`)라서 의존성을 늘리지 않으려고 기존 파서를 재사용했다.
- 저장 전에 검증을 거친 값이라 읽기 실패는 사용자 잘못이 아니라 **데이터 이상**이다. 그래서 `IllegalStateException`(500)으로 바꾼다.

3-8. 접수원: ZoneApiController

```
@RestController
@RequestMapping("/api/v1/manage/zones")
@RequiredArgsConstructor
public class ZoneApiController {

    private final ZonePreviewService zonePreviewService;

    @PostMapping("/preview")
    public ZonePreviewResponse preview(@Valid @RequestBody ZonePreviewRequest request) {
        return zonePreviewService.preview(request.boundary());
    }
}
```

접수원은 **요청을 받아 창구장에게 넘기기만** 한다. 비즈니스 판단은 없다(팀 패키지 규칙). `@Valid @RequestBody`는 JSON 본문을 `ZonePreviewRequest`로 받고 `@NotNull` 검사를 먼저 돌린다.

개정 반영: 초안에는 구역 전용 임시 처리기 `ZoneApiExceptionHandler`(`@RestControllerAdvice(assignableTypes = ZoneApiController.class)`)가 있었다. **현재는 삭제**되고 공통 오류 처리가 `ErrorCode` + `BusinessException`으로 400 응답을 만든다.

4. 테스트 9개

테스트 샘플은 #7의 `regions.geojson`을 그대로 쓴다. 삼청동(경도 127.00~127.05, 위도 37.60~37.65) 안에 **0.002° × 0.002° 사각형**을 그려서 실험한다. 실제 크기는 위도 방향 약 222m, 경도 방향 약 176m이고 넓이는 약 39,190㎡다.

```
@BeforeEach
void loadRegions() {
    jdbcTemplate.update("UPDATE region SET parent_id = NULL");
    jdbcTemplate.update("DELETE FROM region");
```

```
    regionLoadService.load(REGION_FILE);        // #7 기능으로 테스트 데이터를 만든다
}

@Test
void preview_knownSquare_areaWithinOnePercent() {
    ZonePreviewResponse response = zonePreviewService.preview(json(SAMCHEONG_SQUARE));
    assertThat(response.areaM2()).isCloseTo(SAMCHEONG_SQUARE_AREA, within(SAMCHEONG_SQUARE_AREA * 0.0
1));
}
```

테스트	증명하는 것
`preview_knownSquare_areaWithinOnePercent`	**완료 기준**: 약 39,190㎡ 사각형의 면적 오차가 1% 미만
`preview_returnsContainingDong`	사각형이 삼청동 안이라 `dongName`이 "삼청동", `dongId`가 채워짐
`preview_outsideAnyDong_throws`	샘플 지역 밖(경도 128, 위도 35)이면 `OutOfServiceAreaException`
`preview_tooFewPoints_throws`	점이 모자란 도형은 `InvalidPolygonException`
`preview_selfIntersecting_throws`	나비넥타이 모양은 `InvalidPolygonException`
`preview_notPolygon_throws`	Point 같은 다른 종류는 `InvalidPolygonException`
`api_ok`	실제 요청 → 200, `areaM2`·`dongId` 숫자, `dongName` "삼청동"
`api_invalidPolygon`	실제 요청 → 400, `code`가 `INVALID_POLYGON`
`api_outOfServiceArea`	실제 요청 → 400, `code`가 `OUT_OF_SERVICE_AREA`

서비스를 직접 부르는 테스트 6개와, 실제 주소로 요청을 보내는 MockMvc 테스트 3개를 함께 둬서 **계산 로직과 응답 모양을 둘 다** 확인한다.

5. 선배가 물을 만한 질문과 답

질문	답
"그냥 평면 좌표로 넓이 구하면 안 되나?"	위도·경도는 도(°) 단위라 경도 1°의 길이가 위도마다 다르다. 위도로 보정해 미터로 바꾼 뒤 넓이를 구했고, 약 39,190㎡ 사각형으로 오차 1% 미만을 테스트했다.
"왜 내부 점의 위도 하나로 전체를 계산하나?"	명세 공식이 그렇다. 구역이 수백 m 크기라 위도 차이가 작아 오차가 1% 미만이다. 수 km 이상 큰 구역에는 맞지 않는다.
"왜 도형 전체가 아니라 안쪽 점 하나로 동을 정하나?"	명세가 "내부 점이 들어가는 DONG 찾기"라고 정했다. 대신 두 동에 걸쳐도 통과한다는 한계가 있다.
"`GeoJsonReader`를 왜 안 썼나?"	별도 모듈(`jts-io-common`)이라 의존성이 늘어난다. #7의 파서를 재사용했다.
"#7 파일을 왜 고쳤나?"	`readGeometry()` 공개 메서드 한 개만 추가했고 기존 동작은 바꾸지 않았다.
"임시 오류 핸들러는 왜 지웠나?"	첫 CI가 실패했다. dev에 공통 오류 처리가 병합되면서 응답 모양이 달라졌기 때문이다. `ErrorCode`와 `BusinessException`으로 교체했다.
"`@Transactional(readOnly = true)`는 왜?"	이 서비스는 읽기만 하고 저장하지 않아서다.

6. 명세가 정한 것 vs 내가 정한 것

- **명세가 정함**: 다각형 검증(점 3개 이상, 자기 교차 없음, JTS isValid), 내부 점이 들어가는 DONG, 넓이 공식, 응답 `{areaM2, dongId, dongName}`, 오류 `INVALID_POLYGON` / `OUT_OF_SERVICE_AREA`
- **내가 정함**: 넓이를 소수 첫째 자리로 반올림, Polygon만 허용(MultiPolygon 요청은 거부), 동 찾기를 전체 동 순회로 구현, 관리자 권한 검사 생략

7. 명세 대조 (FNC-A-013)

명세 항목	구현	확인
다각형 검증	`ZonePolygonReader`	`preview_tooFewPoints_throws`, `preview_selfIntersecting_throws`
내부 점이 들어가는 DONG 찾기	`getInteriorPoint()` + `findDongContaining()`	`preview_returnsContainingDong`
위도 보정 평면 넓이	`ZoneAreaCalculator`	`preview_knownSquare_areaWithinOnePercent`
`{areaM2, dongId, dongName}` 반환	`ZonePreviewResponse`	`api_ok`
잘못된 다각형 400	`InvalidPolygonException`	`api_invalidPolygon`, `preview_notPolygon_throws`
서비스 지역 밖 400	`OutOfServiceAreaException`	`preview_outsideAnyDong_throws`, `api_outOfServiceArea`
권한: 관리자 이상	**미구현** (보안 설정 없음)	R1